Aufgabe 2.2: Künstliche Testbilder erzeugen

Erklärung zum Skript `Aufgabe2_2.m`, Teilaufgaben a) und c)

Allgemeines zum Format

- `%[text] ...`: Text, den MATLAB im Live Editor formatiert anzeigt (Überschriften, Beschreibungen). `#` ergibt eine große, `##` eine kleinere Überschrift.
- `%%`: teilt das Skript in Abschnitte. Jeden Abschnitt kann man einzeln ausführen.
- `%[appendix]...` am Ende: Einstellungen des Live Editors (z. B. Bilder rechts neben dem Code). Hat mit der Aufgabe nichts zu tun.

a) Zufällige Grauwerte

Gleichverteilung

```
gleich = uint8(randi([64 255], 100, 100));
```

- `randi([64 255], 100, 100)` erzeugt eine **100×100-Matrix** aus **ganzen Zufallszahlen** zwischen 64 und 255. Jede Zahl ist **gleich wahrscheinlich**, daher „gleichverteilt“.
- Jede Zahl ist ein Pixel, ihr Wert ist der Grauwert: 0 = schwarz, 255 = weiß.
- `uint8(...)` wandelt in den Datentyp **uint8** um (ganze Zahlen 0…255). Das ist wichtig, weil `imshow` den Datentyp auswertet:
  - **uint8**: 0 = schwarz, 255 = weiß. Richtig.
  - **double** (Standardtyp): 0 = schwarz, **1 = weiß**. Alles über 1 wäre weiß, das Bild wäre komplett weiß.
- Weil der kleinste Wert 64 ist, gibt es **kein echtes Schwarz**, nur Dunkelgrau bis Weiß.

Normalverteilung

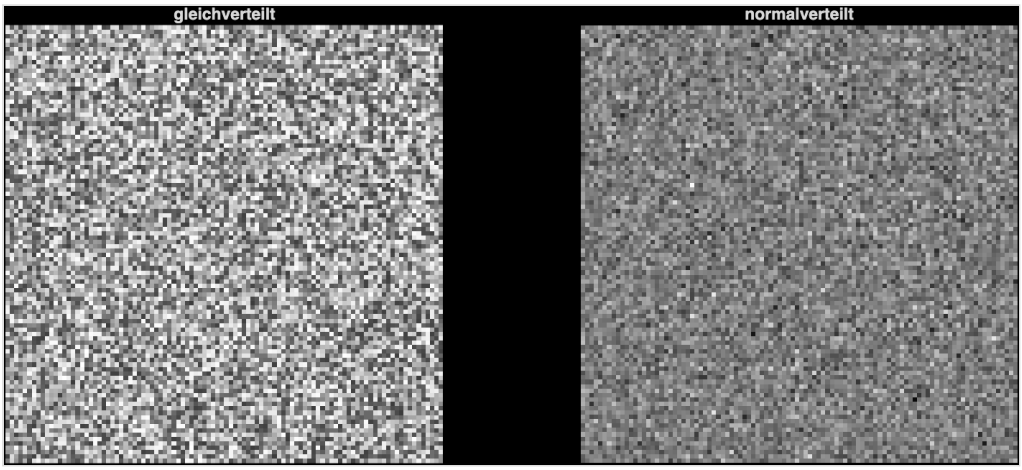
```
mu    = 128;
sigma = 30;
normal = uint8(mu + sigma * randn(100, 100));
```

- `randn(100,100)` liefert Zufallszahlen aus der **Standardnormalverteilung** (Gaußsche Glockenkurve) mit **Mittelwert 0** und **Standardabweichung 1**. Die Werte sind Kommazahlen wie –0.3, 1.2 oder –2.1.
- `sigma * randn` streckt die Glocke: Standardabweichung 30, **Varianz** = sigma² = 30² = 900.
- `mu + ...` verschiebt die Glocke, sodass sie um 128 (mittleres Grau) liegt.
- Faustregel: ca. **68 %** der Pixel liegen zwischen mu – sigma und mu + sigma (98…158), ca. **95 %** innerhalb ±2·sigma (68…188).
- `uint8(...)` macht hier zwei Dinge:
  1. **Runden** auf ganze Zahlen (131.7 → 132).
  2. **Abschneiden** (Sättigung): Werte < 0 werden 0, Werte > 255 werden 255.

Anzeigen

```
figure
subplot(1,2,1)
imshow(gleich)
title('gleichverteilt')
subplot(1,2,2)
imshow(normal)
title('normalverteilt')
```

- `figure` öffnet ein neues Fenster.
- `subplot(1,2,1)` teilt das Fenster in **1 Zeile × 2 Spalten** und wählt Feld **1** (links). `subplot(1,2,2)` ist das rechte Feld.
- `imshow` zeigt die Matrix als Bild, `title` setzt die Überschrift.



Gleichverteilt [64;255] (links) und normalverteilt mit mu = 128, sigma = 30 (rechts)

Warum sehen die Bilder unterschiedlich aus? Bei der Gleichverteilung kommt jeder Wert von 64 bis 255 gleich oft vor, das Bild wirkt **kontrastreich** und hell. Bei der Normalverteilung liegen die meisten Werte **nahe bei 128**, das Bild wirkt **gleichmäßiger grau**.

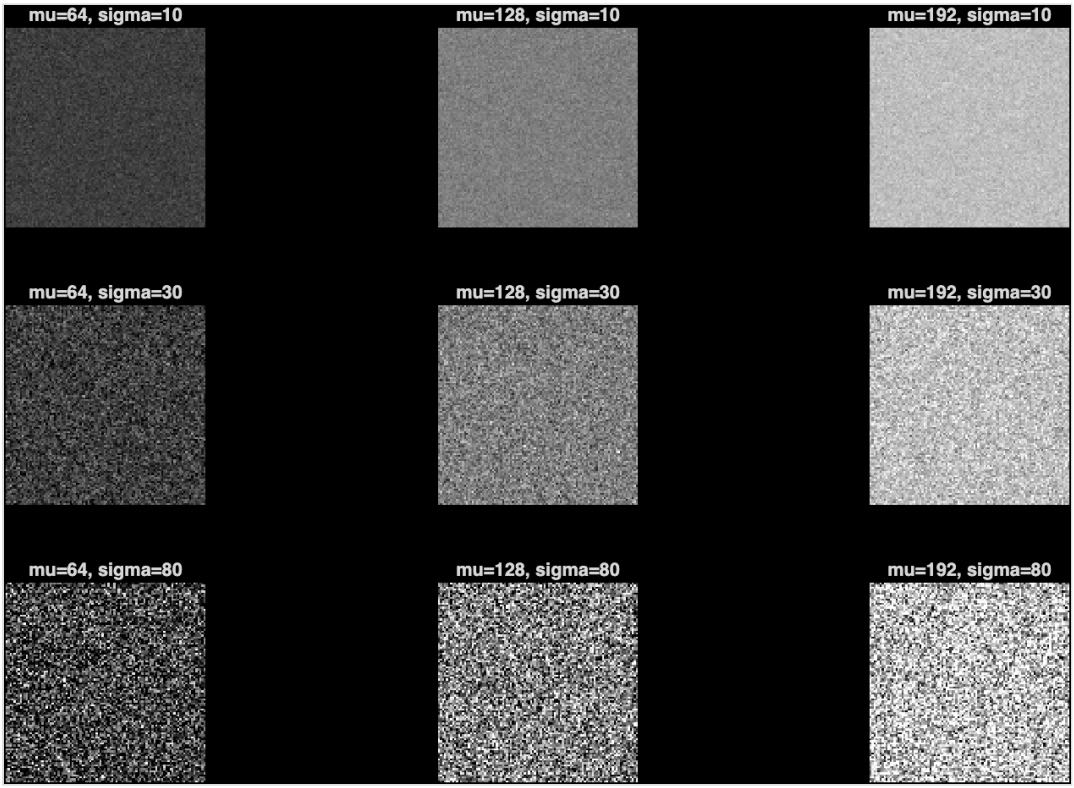
Mittelwert und Varianz variieren

```
mus    = [64 128 192];
sigmas = [10 30 80];
```

Drei Mittelwerte (dunkel, mittel, hell) und drei Standardabweichungen (wenig, mittel, viel Streuung) ergeben **3 × 3 = 9 Kombinationen**.

```
figure
k = 1;
for i = 1:3
    for j = 1:3
        bild = uint8(mus(j) + sigmas(i) * randn(100, 100));
        subplot(3, 3, k)
        imshow(bild)
        title(['mu=' num2str(mus(j)) ', sigma=' num2str(sigmas(i))])
        k = k + 1;
    end
end
```

- **Zwei verschachtelte Schleifen:** die äußere (`i`) wählt **sigma** und damit die **Zeile**, die innere (`j`) wählt **mu** und damit die **Spalte**.
- `k` zählt von 1 bis 9 und gibt das Feld im 3×3-Raster an. `subplot` zählt **zeilenweise**: 1 2 3 / 4 5 6 / 7 8 9.
- `num2str(...)` macht aus einer Zahl Text, `['a' 'b']` hängt Texte aneinander, z. B. zum Titel `mu=128, sigma=30`.
- Ablauf: i=1 (sigma=10) mit j=1,2,3 (mu=64,128,192) in die Felder 1,2,3, dann i=2 (sigma=30) in die Felder 4,5,6 usw.



Spalten: mu = 64, 128, 192 · Zeilen: sigma = 10, 30, 80

Beobachtung (laut Aufgabe beschreiben)

Änderung	Effekt	Warum
mu größer	Bild wird **heller**	alle Werte nach oben verschoben
mu kleiner	Bild wird **dunkler**	alle Werte nach unten verschoben
sigma klein	fast **einheitlich grau**, wenig Kontrast	Werte liegen eng beim Mittelwert
sigma groß	**starker Kontrast**, sehr verrauscht	Werte streuen weit
sigma sehr groß (80)	viele **rein schwarze / weiße** Pixel	Werte außerhalb 0…255 werden durch `uint8` abgeschnitten

Unten links (mu = 64, sigma = 80) sieht man das gut: Fast die Hälfte der Werte wäre negativ und wird zu 0 (schwarz).

> **Hinweis:** Jeder Durchlauf erzeugt andere Bilder, weil die Zahlen zufällig sind. Das ist normal.

c) Rauschen mit `imnoise`

```
grau = rgb2gray(imread('Auge.jpg'));
```

- `imread('Auge.jpg')` liest das Farbbild ein: eine **H×B×3-Matrix** (uint8). Die 3 Ebenen sind die Farbkanäle Rot, Grün, Blau.
- `rgb2gray(...)` macht daraus ein **Grauwertbild** (H×B). Die Kanäle werden **gewichtet**: ca. 0,30·R + 0,59·G + 0,11·B, weil das Auge Grün am stärksten wahrnimmt. Das ist der Unterschied zum einfachen Mittelwert aus Aufgabe 1.2a.

`imnoise(bild, 'typ')` fügt einem Bild künstliches Rauschen hinzu. Verwendet werden jeweils die **Standardwerte** (keine zusätzlichen Parameter).

1) Gaußsches Rauschen: A/D-Wandlungsfehler

```
gauss = imnoise(grau, 'gaussian');
```

- Auf **jedes Pixel** wird ein normalverteilter Zufallswert **addiert** (gleiches Prinzip wie in a).
- Standard: Mittelwert 0, Varianz 0,01 auf der Skala 0…1, also ca. **25 Grauwerte** Standardabweichung auf 0…255.
- Das Rauschen ist **überall gleich stark**, egal ob hell oder dunkel.

2) Poisson-Rauschen: Dunkelstrom / Photonenrauschen

```
poisson = imnoise(grau, 'poisson');
```

- Licht besteht aus einzelnen Photonen, deren Anzahl zufällig schwankt.
- Das Rauschen **hängt vom Pixelwert ab**: je heller, desto größer die absolute Schwankung.
- Im Bild am **schwächsten** sichtbar, da kein extra Parameter, sondern direkt aus den Bildwerten berechnet.

3) Salt & Pepper: defekte Pixel

```
saltpep = imnoise(grau, 'salt & pepper');
```

- Standardmäßig werden zufällig **5 % der Pixel** komplett **weiß (255, Salz)** oder **schwarz (0, Pfeffer)**.
- Alle anderen Pixel bleiben **unverändert**: einzelne Punkte, der Rest bleibt scharf.

4) Speckle: Radar- und Ultraschallbilder

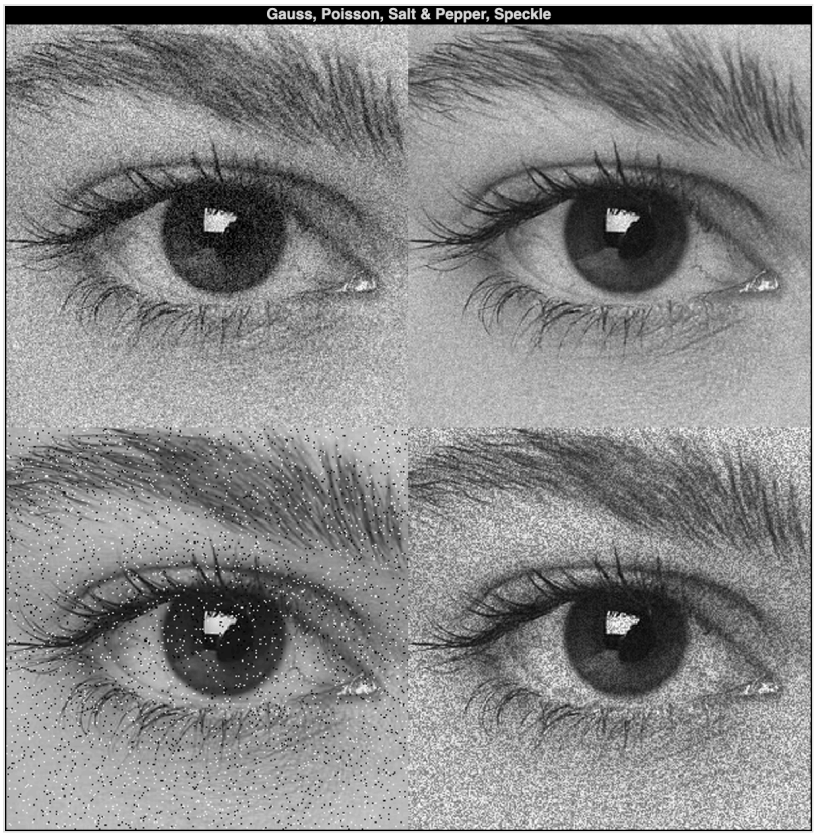
```
speckle = imnoise(grau, 'speckle');
```

- Das Rauschen wird **multipliziert** statt addiert: `neu = bild + n·bild`, n zufällig (Varianz 0,04).
- **Helle Bereiche rauschen stark, dunkle kaum**: Die Pupille bleibt fast rauschfrei, die helle Haut ist stark körnig.

Gemeinsam anzeigen

```
figure
montage({gauss, poisson, saltpep, speckle}, 'Size', [2 2])
title('Gauss, Poisson, Salt & Pepper, Speckle')
```

- `{ ... }` ist ein **Cell-Array**, eine Liste, die mehrere Bilder zusammenfasst.
- `montage` setzt alle Bilder zu **einem großen Bild** zusammen.
- `'Size', [2 2]` = **2 Zeilen × 2 Spalten**, zeilenweise gefüllt.



Oben: Gauss (links), Poisson (rechts) · Unten: Salt & Pepper (links), Speckle (rechts)

Mögliche Fragen bei der Präsentation

Frage	Antwort
Warum `uint8`?	Damit `imshow` 0…255 richtig darstellt. Werte werden gerundet und auf 0…255 begrenzt.
Was macht `mu + sigma*randn`?	`randn` hat Mittelwert 0 und Standardabweichung 1. Mit sigma wird gestreckt, mit mu verschoben.
Varianz vs. Standardabweichung?	Varianz = sigma². Bei sigma = 30 ist die Varianz 900.
Gauß vs. Speckle?	Gauß wird **addiert** und ist überall gleich stark. Speckle wird **multipliziert** und ist in hellen Bereichen stärker.
Was ist besonders an Salt & Pepper?	Nur wenige Pixel sind betroffen, aber komplett: ganz schwarz oder ganz weiß.